



Algoritmi e strutture dati

ASD2023_I_Programmazione [AULA]



YIMING HOU

297226

Iniziato martedì, 7 febbraio 2023, 09:38

Terminato martedì, 7 febbraio 2023, 11:18

Tempo impiegato 1 ora 40 min.

Informazione

Attenzione!

Indicare quale tipologia di esame si intenda svolgere rispondendo alla domanda a scelta multipla seguente (domanda 1).

Si noti che è possibile leggere tutte o parte delle domande prima di effettuare la scelta e rispondere alla domanda 1.

Le domande dalla 2 alla 6 sono dedicate all'esame semplificato (traccia da 12pt).

In particolare:

- la domanda 2 fa parte dell'esercizio da 2 punti.
- la domanda 3 fa parte dell'esercizio da 4 punti.
- le domande 4,5,6 fanno parte dell'esercizio da 6 punti.

Le domande dalla 7 in poi sono dedicate all'esame completo (traccia da 18pt).

Un apposito separatore fa da intervallo tra le domande del compito da 12pt e le domande del compito da 18pt.

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti)

- se non indicato diversamente, è consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne
- gli header file riferiti dal codice dovranno essere inclusi nella versione del programma allegata alla relazione
- i modelli delle funzioni ricorsive **non** sono considerati funzioni standard
- consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro VENERDI' 10/02/2023, alle ore 14:00, mediante caricamento su Portale
- le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale
- QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERRÀ APPLICATA UNA PENALIZZAZIONE. Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto durante l'appello. Non verranno corretti i compiti di cui non sarà stata inviata la relazione nei tempi stabiliti.

Domanda 1

Completo

Non valutata

Indicare quale tipologia di esame si intende svolgere.

- ☒ (a) 12 Punti - Semplificato
- ☐ (b) 18 Punti - Completo

Domanda 2

Completo

Punteggio max.: 2,00

Sia data una matrice M di dimensione $r \times c$ contenente elementi interi.
Scrivere una funzione che generi una matrice M' di dimensione $r \times c$ derivata da M sostituendo il valore di ogni cella con la media di tutti i suoi vicini (diagonali incluse) e la cella stessa.
Per gli angoli la media sarà quindi tra 4 valori, per le celle lungo i bordi (angoli esclusi) tra 6

elementi e per tutte le altre celle tra 9 elementi.

La matrice M' sia allocata dentro alla funzione.

Completare opportunamente il prototipo in modo che la nuova matrice sia disponibile al chiamante.

```
void f(int **M, int r, int c, ...);
```

$$M = \begin{Bmatrix} 1 & 2 & 1 \\ 2 & 0 & 2 \\ 1 & 2 & 1 \end{Bmatrix} \quad M' = \begin{Bmatrix} 1.25 & 1.33 & 1.25 \\ 1.33 & 1.33 & 1.33 \\ 1.25 & 1.33 & 1.25 \end{Bmatrix}$$

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```

void f(int **M, int r, int c, int ***NEW) {
float **NM;
int i,j,k=r*c, n,s, startn, finn, starts, fins;
int su, giu;
float t;
for(i=0; i<r; i++){
    *NM=malloc(c*sizeof(float));
}
NM=malloc(r*sizeof(float));
for(i=0; i<r; i++){
    k=r*c; su=1; giu=1; dx=1; sx=1;
    for(j=0; j<c; j++){
        startn=i-1; starts=j-1; finn=i+1; fins=j+1;
        if(i==0){
            k=k-c;
            su=0;
            startn=0;
        }
        if(i==c-1){k=k-c; giu=0; finn=i;}
        if(j==r-1){
            if(giu==0 && su==0) k=k-r+2;
            else if(giu==0 || su ==0) k=k-r+1;
            else k=k-r;
            fins=j;
        }
        if(j==0){
            if(giu==0 && su==0) k=k-r+2;
            else if(giu==0 || su ==0) k=k-r+1;
            else k=k-c;

            starts=0;}
        for(n=startn; n<finn; n++) for( s=starts; s<fins; s++) tmp=tmp+M[n][s];
        NM[i][j]=(float)tmp/(float)k;
    }
}
NM=*NEW;
}

```

Domanda 3

Completo

Punteggio max.: 4,00

Si scriva una funzione wrapper `int f(T t)` (e la relativa funzione ricorsiva) che ricevuto in input un albero n-ario di interi `t` di tipo `T` conti la lunghezza del cammino più lungo contenente solo valori positivi o nulli.

Fornire la definizione del tipo `T` e del tipo nodo al suo intero, come ADT di prima classe e come quasi ADT rispettivamente, indicando esplicitamente la divisione in moduli adottata.

Per rappresentare l'albero richiesto, ogni nodo tiene traccia dei figli mediante un vettore di puntatori a nodo e il numero di figli.

Non è ammesso l'uso di funzioni di libreria.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

T.h

```
typedef struct node {int ogg; struct node *link; struct node *z} *nodo;
typedef struct ti *T;
```

T.c

```
struct ti{nodo root; nodo  figli[2]; int n}
```

```
int fu(nodo t, int *max, int tmax){
int a, b;
if(t==z)
    return tmax-1;
if(t->root->ogg>=0){
a=fu(t->figli[0], max, tmax+1);
b=fu(t->figli[1], max, tmax+1);}
else{
a=fu(t->figli[0], max, tmax;
b=fu(t->figli[1], max, tmax;
}
if(a>b)
    tmax=a;
if(b>=a)
    tmax=b;

return tmax;
}
```

```
int f(T t){
int max, tm;
    return fu(t->link, &max, 0 );

}
```

```
//T.c
//scrivere qui il codice...
...
int f(T t) { ... }
```

Domanda 4

Completo

Punteggio max.: 6,00

Una matrice M quadrata di dimensione $N \times N$ rappresenta le distanze dirette tra N città. Ogni cella contenente un valore positivo d indica che la coppia di città (i,j) dista d . Un valore pari a zero indica l'assenza di una connessione diretta tra le due città. Scrivere una funzione ricorsiva in grado di suddividere le città nel minor numero di gruppi possibili facendo in modo che tutte le città in un gruppo siano mutualmente raggiungibili (direttamente o transitivamente) attraversando al massimo una città intermedia. È ammesso che ci siano gruppi composti da una singola città.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
//scrivere qui il codice...
int findpath(int **M, int group, int *v, int start, int in){
    int i;
    for(i=0; i<start; i++){
        if(M[in][v[i]]>0){
            if(M[v[i]][group]>0);
            return 1;}
    }
    return 0;
}
int control(int **M, int group, int tsol, int N, int start){
    int i,j;
```

```

for(i=0; i<start, i++){
if( M[tsol[i]][group]==0){
    if(!findpath(M, group, tsol, start, tsol[i]))
        return 0; //se c'è un singolo elemento non connesso
    }
}
}
return 1;
}
int disp(int pos, int **M, int *sol, int *tm, int mark[N], int start, int k){
int i;
if(pos>=k)
    for(i=0; i<N; i++){
        if(mark[i]==-1)//se non ho preso ogni città
            return 0;
    }
    return 1;
for(i=0; i<N; i++){
    if(control(M, i, tm, N, start) && mark[i]==-1){
        tm[i]=pos; mark[i]=1;
        if(disp(pos, M, sol, tm, mark, start+1, k)) return 1;}
    }
    else if(mark[i]==-1){
        mark[i]=1;
        if(disp(pos+1, M, sol, tm, mark, 0, k); mark[i]=1;) return 1;}
    mark[i]=-1;
}

void wrap(int **M, int N){
int i, flag=0;
int mark[N];
int *sol=malloc(N*sizeof(int)), *tm=malloc(N*sizeof(int));
for(i=0; i<N; i++) { mark[i]=-1;}
for(i=0; i<N && !flag; i++)
    if(disp(0, M, sol, tm, mark, 0, i )) flag=1;
printf("%d", i);
//sol vettore rindondante da togliere
}

```

Domanda 5

Completo

Non valutata

Si giustifichi la scelta del modello combinatorio adottato.

disposizioni semplici con k crescente

Non è possibile prendere una stessa città più volte ma l'ordine conta (quale città è in quale gruppo)

pos rappresenta il numero di gruppi presenti

start rappresenta il numero di elementi dentro un gruppo

Domanda 6

Completo

Non valutata

Si descrivano i criteri di pruning adottati o il motivo della loro assenza.

pruning in scelta, se si rispettano di criteri di mutua raggiungibilità

Informazione**PAGINA DI INTERMEZZO**

La prova da 12 punti termina prima di questa pagina di intermezzo.

La prossima domanda rappresenta l'inizio della prova da 18 punti.

Informazione

Descrizione del problema

In un noto videogioco, il giocatore deve superare delle prove di *hacking* inserendo delle sequenze di token esadecimali di due caratteri in un buffer di lunghezza fissa per ottenere vari bonus. Ai fini del problema, si consideri la seguente descrizione del mini-gioco.

I token vengono scelti da una matrice quadrata di dimensione $N \times N$. Ogni token della matrice può essere usato una singola volta, ma nella matrice possono apparire dei duplicati.

La scelta dei token avviene alternando una serie di movimenti per riga e per colonna, partendo sempre con una scelta iniziale sulla prima riga. I movimenti possono interessare un qualsiasi token lungo una certa riga/colonna, indipendentemente dalla distanza dalla posizione corrente. Non sono mai ammessi movimenti in diagonale.

Lo schema di movimento/scelta è quindi simile alla sequenza a seguire:

- Il primo token (scelta di indice zero) viene sempre scelto sulla prima riga
- Il secondo token (scelta di indice uno) viene scelto nella colonna in cui si sia scelto il token precedente
- Il terzo token (scelta di indice due) viene scelto nella riga in cui si sia scelto il token precedente
- E così via, alternando colonna/riga nelle mosse a seguire...

I bonus ottenuti dal giocatore sono associati a delle stringhe obiettivo, composte anch'esse da token esadecimali. Per semplicità, si assuma che il bonus sia un valore intero positivo.

Il giocatore ottiene tutti i bonus la cui stringa associata appaia come sottostringa nel buffer riempito dal giocatore.

Il programma riceve in input la lunghezza L del buffer, un file contenente la matrice di gioco `grid.txt` e un file contenente la lista di bonus disponibili `bonus.txt`.

Il file `grid.txt` riporta sulla prima riga la dimensione N della matrice e a seguire N righe di N token esadecimali separati da un singolo spazio.

Il file `bonus.txt` riporta sulla prima riga il numero di B bonus disponibili e a seguire B righe nel formato:

`<numero_token> <valore_bonus> <sequenza_di_token>`

Esempi

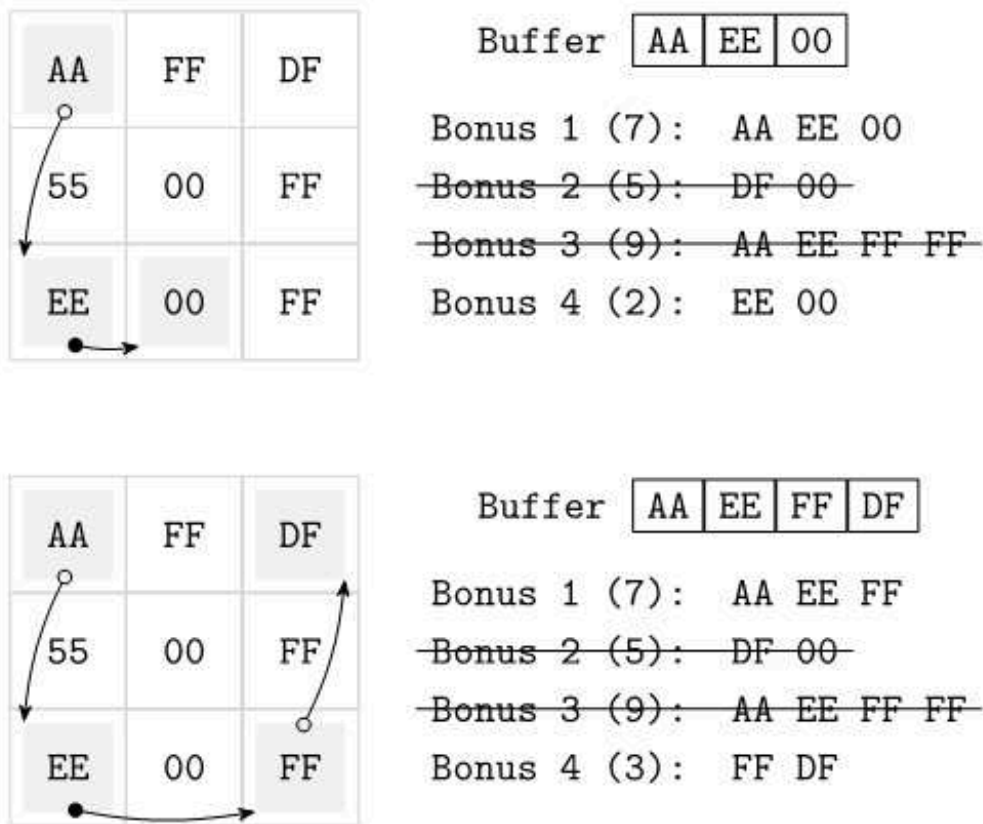
Si faccia riferimento all'immagine proposta a seguire, assumendo che per il primo esempio si abbia $L=3$ e per il secondo esempio si abbia $L=4$.

Con i dati di esempio (Es. 1, sopra) il primo e quarto bonus sono entrambi ottenibili introducendo la sequenza AA EE 00 (che corrisponde alla selezione dei token di posizione $[0,0]$, $[2,0]$, $[2,1]$), che rappresenta anche la sequenza a valore massimo che si possa introdurre. Il secondo bonus non è ottenibile poiché non c'è nessuna sequenza di movimenti, indipendentemente dalla lunghezza del buffer, che permetta di introdurre DF 00 nel buffer stesso. Il terzo bonus non è ottenibile poiché il buffer ha lunghezza $L=3$. Con buffer più lunghi sarebbe stato possibile ottenere quel bonus poiché la sequenza corrispondente quantomeno esiste come sequenza di mosse valide nella griglia ($[0,0]$, $[2,0]$, $[2,2]$, $[1,2]$).

Con i dati di esempio (Es. 2, sotto), il buffer di lunghezza quattro permetterebbe di completare tutte le sequenze, individualmente o meno, sempre con l'eccezione della seconda che non è realizzabile data la griglia di input. Per il valore dei bonus, conviene introdurre AA EE FF DF che

copre il primo e il quarto bonus anziché AA EE FF FF che coprirebbe solamente il terzo, con resa inferiore.

Nelle immagini di esempio, si sono distinte mosse obbligate per riga (colonna) con il pallino pieno (vuoto) come marker di inizio della freccia.



Richieste del problema

A seguire una sintesi delle richieste del problema. Per ogni richiesta si troverà una domanda dedicata nelle sezioni a seguire con una descrizione più dettagliata per le richieste.

Strutture dati

Definire opportune strutture dati per rappresentare la collezione di bonus, la griglia di gioco, e quanto sia ritenuto necessario alla soluzione dei problemi di verifica e di ricerca. Acquisire i dati nelle strutture definite.

Problema di verifica

Data una proposta di soluzione, valutare che questa rispetti i vincoli di movimento esposti in relazione ai contenuti della matrice di gioco. In caso affermativo, valutare il valore della somma dei bonus ottenibili.

Problema di ottimizzazione

Identificare la scelta di token con cui riempire il buffer che massimizzi la somma dei bonus ottenibili.

Domanda 7

Risposta non data

Non valutata

Strutture dati e acquisizione

Definire opportune strutture dati per rappresentare la collezione di bonus, la griglia di gioco, e quanto sia ritenuto necessario alla soluzione dei problemi di verifica e di ricerca.

Scrivere qui la definizione e implementazione delle strutture dati reputate necessarie a modellare le informazioni del problema secondo quanto richiesto e le funzioni di acquisizione dei dati stessi. In caso di organizzazione delle strutture dati su più file, indicare esplicitamente il modulo di riferimento.

Domanda 8

Risposta non data

Non valutata

Problema di verifica

Data una sequenza di scelte sulla griglia, valutare che questa rispetti i vincoli di movimento esposti e determinare la composizione del buffer. In caso di sequenza valida, valutare il valore della somma dei bonus ottenibili. Il formato dei dati in input per questa richiesta è a discrezione del candidato, che è tenuto a fornirne anche una breve descrizione.

Domanda 9

Risposta non data

Non valutata

Problema di ricerca e ottimizzazione

Identificare la scelta di token con cui riempire il buffer che massimizzi la somma dei bonus ottenibili.
